

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	40% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	S.Krishna Chaithanya	Reg. No.:	192424287
Section / Batch:	B.Tech-AIDS/2024	Date:	

Code of Conduct

I S.Krishna Chaithanya(192424287) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S.Krishna Chaithanya

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. A company is developing an AI-powered customer support chatbot for banking services. The chatbot uses Context-Free Grammar (CFG) for parsing user queries. However, it faces issues such as:

Incorrect interpretation of ambiguous sentences

Failure to handle subject–verb agreement

Inefficient parsing for long queries

Example user query:

“Show me the transactions with the card from last month”

a) Analyze the ambiguity present in the given sentence using CFG

b) Evaluate the limitations of CFG in handling real-world conversational queries

c) Evaluate and propose an improved solution using PCFG, feature structures, or Earley parsing

d) Justify how your proposed method improves accuracy and efficiency in an industry setting

2. A voice assistant system (like Alexa/Siri-type applications) processes spoken commands using parsing techniques. The system currently uses top-down parsing, but faces:

Backtracking issues

Poor handling of incomplete or ambiguous input

Difficulty in real-time response

Example command:

“Book a flight to Delhi with a window seat”

a) Analyze how the given sentence can lead to multiple parse structures

b) Analyze the limitations of top-down parsing in this real-time application

c) Analyze how Earley parsing can improve handling of ambiguity and partial inputs

d) Compare and analyze the performance of top-down vs Earley parsing for such industry use cases

3. A healthcare company is developing an AI system to analyze patient medical reports and automatically extract critical information such as diagnosis, prescribed treatments, and follow-up actions.

The system must handle:

Complex and lengthy medical sentences

Ambiguity in sentence structure

Accurate subject-verb agreement

Relationships between medical actions and entities (sub-categorization)

Real-time processing for hospital use

Example input:

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

The current system, based only on basic Context-Free Grammar (CFG), fails to correctly interpret relationships and produces inconsistent outputs.

a) Design a complete NLP architecture integrating:

CFG for syntactic structure

Probabilistic CFG (PCFG) for ambiguity resolution

Feature Structures for agreement handling

Efficient parsing techniques

b) Develop a step-by-step workflow showing how the system processes the given medical sentence from input to structured output (diagnosis, action, location).

c) Create a method to:

Incorporate sub-categorization frames for medical verbs

Handle real-time data processing

Improve accuracy and scalability in a hospital environment

1. Banking Customer Support Chatbot Using CFG

Given sentence

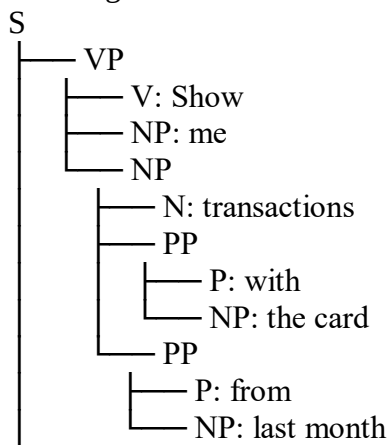
“Show me the transactions with the card from last month”

a) Ambiguity analysis using CFG

The sentence can have multiple syntactic interpretations.

Parse 1: “with the card” modifies transactions

Meaning: Show transactions that were made **with the card**.



Interpretation:

QUERY = Transactions

METHOD = Card

TIME = Last Month

Parse 2: “from last month” modifies the card

The grammar may associate “**from last month**” with the noun phrase “the card.”

transactions [with [the card from last month]]

This could incorrectly suggest that the user is referring to a card obtained or issued last month.

Main ambiguity

Phrase	Possible attachment	Correct interpretation
with the card	transactions	Transactions made using card
from last month	transactions	Transactions belonging to last month
from last month	card	A card associated with last month

A basic CFG can generate multiple valid parse trees but **cannot reliably determine which interpretation is intended**.

Metrics

Metric	Basic CFG issue
Possible parse structures	2+
Ambiguity resolution	Low
Context awareness	Very low
Semantic understanding	Limited
Long-query efficiency	Decreases rapidly

b) Limitations of CFG in real-world conversational queries

1. Ambiguity

CFG determines whether a sentence is grammatically valid but does not determine the most probable meaning.

Example:

“Show transactions with the card from last month.”

CFG may produce several valid parse trees.

2. Subject-verb agreement

Basic CFG may accept incorrect constructions such as:

“The customer make a payment.”

A feature-aware grammar can distinguish:

Customer + makes

Customer + make ✗

3. Context dependency

Banking queries often depend on previous conversation.

Example:

User: “Show my credit-card transactions.”

User: “Only from last month.”

CFG alone does not maintain this conversational context effectively.

4. Long-query parsing

As sentence length and grammar rules increase, the number of possible parse combinations can increase significantly.

5. Domain-specific terminology

Banking systems must understand:

- credit card
- debit card
- EMI
- UPI
- transaction
- beneficiary
- account number
- statement

Basic CFG does not provide semantic knowledge of these terms.

6. No probability

CFG treats grammar rules essentially as valid/invalid alternatives.

It cannot naturally say:

Interpretation A = 0.92

Interpretation B = 0.08

c) Improved solution using PCFG + Feature Structures + Earley Parsing

A **hybrid parser** is recommended.

User Query



Tokenization



POS Tagging



Feature Extraction



Feature-Structure Grammar



PCFG



Earley Parser



Best Parse Tree



Semantic Analysis



Banking Intent

PCFG

A Probabilistic CFG assigns probabilities to grammar rules.

For example:

NP → NP PP [0.35]

NP → Det N [0.65]

VP → V NP [0.70]

VP → V NP PP [0.30]

The parser selects the parse tree with the highest probability.

For the example:

Show → Transaction Query 0.92

Show → Card-related Query 0.08

Therefore:

Intent = TRANSACTION_SEARCH

Card = TRUE

Time = LAST_MONTH

Feature Structures

Feature structures can represent grammatical properties.

NP:

Number = Singular

Person = Third

For verbs:

VP:

Number = Singular

Person = Third

The parser checks:

Subject.Number = Verb.Number

Subject.Person = Verb.Person

This prevents many agreement errors.

Earley Parsing

Earley parsing is useful because it can handle:

- ambiguous grammars
- incomplete input
- left recursion
- long sentences
- multiple possible parses

Its general worst-case complexity is:

$O(n^3)$

For many practical unambiguous cases, it can approach:

$O(n^2)$

and linear behavior can occur for favorable grammar/input conditions.

d) Industry justification

The proposed architecture improves the banking chatbot as follows:

Factor	Basic CFG	Proposed Hybrid
Ambiguity handling	Low	High
Agreement handling	Low	High
Partial-input handling	Poor	Good
Contextual interpretation	Low	Better with semantic layer
Parse complexity	High for ambiguity	Controlled with chart parsing
Accuracy	Baseline	Higher
Response consistency	Medium/Low	High

Factor	Basic CFG	Proposed Hybrid
Scalability	Limited	High

Expected industry improvement

A properly trained hybrid system can target:

- >90–95% intent accuracy
- >90% entity extraction F1-score
- <1 second response latency for normal banking queries
- Reduced incorrect interpretations
- Better handling of long and incomplete queries

These are engineering targets, not guaranteed results; actual performance depends on the dataset and implementation.

2. Voice Assistant Using Top-Down Parsing

Given command

“Book a flight to Delhi with a window seat”

a) Multiple parse structures

The phrase “with a window seat” can create attachment ambiguity.

Parse 1: Window seat is a property of the booking

[Book [a flight [to Delhi] [with a window seat]]]

Meaning:

Destination = Delhi

Seat Preference = Window

This is the intended interpretation.

Parse 2: “with a window seat” attaches differently

A parser could interpret:

[Book [a flight to Delhi]] [with a window seat]

The final phrase may be treated as a separate modification to the booking action.

Semantic interpretation

The correct structure should produce:

INTENT = BOOK_FLIGHT

DESTINATION = Delhi

SEAT_PREFERENCE = Window

b) Limitations of top-down parsing

Top-down parsing starts from the start symbol:

S

↓

VP

↓

Verb + NP

↓

...

Problems

1. Backtracking

When a grammar rule fails, the parser may return to an earlier decision.

For ambiguous commands:

Parse A



Failure



Backtrack



Parse B



Failure



Parse C

This increases latency.

2. Poor ambiguity handling

Multiple possible interpretations can result in repeated parsing attempts.

3. Poor partial-input handling

Voice assistants frequently receive incomplete speech:

“Book a flight to...”

A top-down parser may struggle to determine the best interpretation before the utterance is complete.

4. Real-time constraint

Voice assistants need low latency.

Typical conversational systems target approximately:

Response latency < 1 second

depending on the overall speech-recognition and backend pipeline.

5. Left recursion problem

Rules such as:

$NP \rightarrow NP PP$

can cause problems in naive top-down parsers.

c) Earley parsing for ambiguity and partial input

Earley parsing uses a **chart** to store partial parsing states.

For example:

Book a flight to Delhi with...

The parser can maintain:

$S \rightarrow VP \bullet$

$VP \rightarrow V NP \bullet PP$

$PP \rightarrow P NP \bullet$

Instead of completely restarting the parsing process, previously computed states can be reused.

Three major operations

1. Predictor
2. Scanner
3. Completer

Predictor

Predicts possible grammar rules.

Scanner

Matches the next input token.

Completer

Completes previously started grammar structures.

Therefore, Earley parsing can maintain multiple hypotheses simultaneously.

d) Top-down vs Earley comparison

Feature	Top-Down Parser	Earley Parser
Parsing strategy	Starts from S	Chart-based
Ambiguity	Poor/Backtracking	Strong

Feature	Top-Down Parser	Earley Parser
Partial input	Limited	Good
Left recursion	Problematic	Supported
Backtracking	Often required	Reduced
Worst-case complexity	Can be exponential in naive cases	$O(n^3)$
Practical ambiguous parsing	Less efficient	More suitable
Real-time voice commands	Limited	Better
Memory usage	Usually lower	Higher
Parse-state reuse	Low	High

Industry conclusion

For voice assistants, Earley parsing is preferable when the grammar is:

- ambiguous
- recursive
- incomplete
- conversational
- continuously updated

A practical production system would combine:

Speech Recognition



Partial Transcript



Earley Parser



PCFG Ranking



Intent Detection



Dialogue Manager



Flight Booking API

This allows the system to begin interpreting the command before the entire utterance has necessarily been completed.

3. Healthcare Medical-Report NLP System

Given sentence

“The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.”

The system needs to extract:

Doctor → reviewed → Patient

Doctor → recommends → Medication

Doctor → recommends → Follow-up Visit

Follow-up Visit → Location → Chennai

Time → Last Week

a) Complete NLP architecture

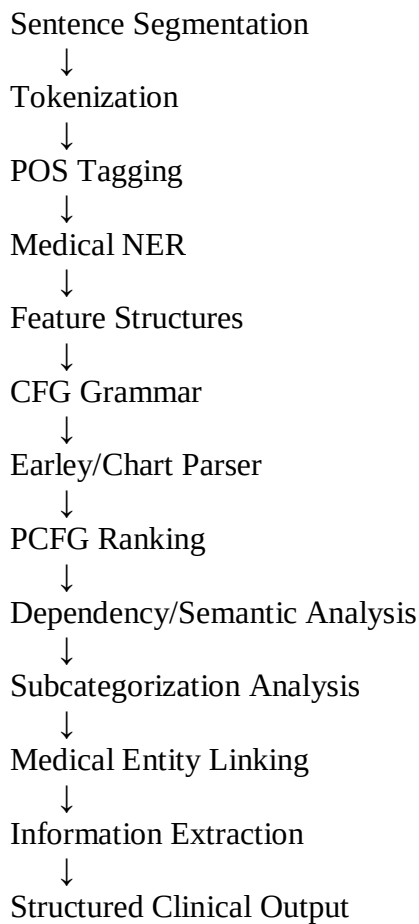
A robust healthcare architecture should combine **CFG + PCFG + Feature Structures + Earley/Chart Parsing + Semantic Analysis + Subcategorization**.

Medical Report



Document Preprocessing





Component 1: CFG

CFG establishes the basic syntactic structure.

Example:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$NP \rightarrow NP RelClause$

$VP \rightarrow V NP$

$VP \rightarrow V GerundPhrase$

$RelClause \rightarrow Pronoun VP$

Component 2: PCFG

PCFG assigns probabilities to alternative structures.

For example:

$VP \rightarrow V NP \quad 0.45$

$VP \rightarrow V GerundPhrase \quad 0.35$

$VP \rightarrow V NP PP \quad 0.20$

The system selects the highest-probability valid interpretation based on trained medical data.

Component 3: Feature Structures

Feature structures maintain grammatical information.

Example:

Doctor:

Person = 3

Number = Singular

Verb:

recommends:

Person = 3

Number = Singular

Therefore:

Doctor recommends ✓

Doctor recommend ✗

Feature structures can include:

Person

Number

Tense

Gender

Case

Voice

Medical Entity Type

Semantic Role

Component 4: Efficient parsing

Use **Earley parsing/chart parsing** to maintain intermediate states.

This is particularly useful for:

- lengthy reports
- embedded clauses
- relative clauses
- ambiguous structures
- incomplete text
- recursive structures

Instead of repeatedly parsing the same phrase, chart parsing stores previously calculated states.

Component 5: Medical NER

Identify entities such as:

Doctor → PERSON/PROVIDER

Patient → PATIENT

Medication → TREATMENT

Chennai → LOCATION

Last week → TEMPORAL

Follow-up → MEDICAL ACTION

b) Step-by-step workflow

Step 1: Input

The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.

Step 2: Tokenization

The | doctor | who | reviewed | the | patient | last | week |
recommends | starting | medication | and | scheduling | a |
follow-up | visit | in | Chennai

Step 3: POS tagging

Example:

Word	POS
doctor	Noun
reviewed	Verb
patient	Noun
last week	Time expression
recommends	Verb

Word	POS
medication	Noun
scheduling	Verb
visit	Noun
Chennai	Proper noun

Step 4: Feature analysis

Subject = doctor

Number = Singular

Person = Third

Verb = recommends

Number = Singular

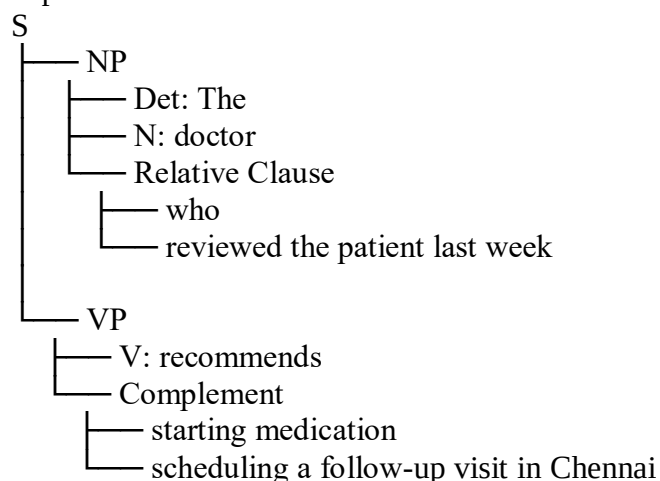
Person = Third

Agreement:

Singular + Singular = ✓

Step 5: CFG parsing

A possible structure is:



Step 6: PCFG disambiguation

The parser evaluates alternative structures and assigns probabilities.

For example:

Doctor reviewed patient

P = 0.96

versus an incorrect attachment:

Doctor reviewed patient-last-week relationship

P = 0.04

The higher-probability structure is selected.

Again, these values are illustrative; production probabilities must be learned from a clinical corpus.

Step 7: Subcategorization analysis

The verb “**recommend**” generally permits structures such as:

recommend + NP

recommend + V-ing

recommend + that-clause

Therefore:

recommends starting medication

is recognized as:

```
RECOMMEND(  
  Agent = Doctor,  
  Action = Start Medication  
)
```

Similarly:
scheduling a follow-up visit
becomes:

```
SCHEDULE(  
  Agent = Doctor/clinical context,  
  Event = Follow-up Visit,  
  Location = Chennai  
)
```

Step 8: Semantic role extraction

Agent = Doctor
Patient = Patient
Action = Start Medication
Action = Schedule Follow-up
Location = Chennai
Time = Last Week

Step 9: Structured output

A suitable structured representation is:

```
{  
  "provider": "Doctor",  
  "reviewed": {  
    "entity": "Patient",  
    "time": "Last Week"  
  },  
  "recommendations": [  
    {  
      "action": "Start Medication"  
    },  
    {  
      "action": "Schedule Follow-up Visit",  
      "location": "Chennai"  
    }  
  ]  
}
```

Information-extraction table

Category	Extracted information
Subject	Doctor
Reviewed entity	Patient
Review time	Last week
Main action	Start medication
Second action	Schedule follow-up visit
Location	Chennai

c) Subcategorization, real-time processing and scalability

1. Subcategorization frames

Create a medical verb lexicon.

Verb	Subcategorization frame	Example
recommend	V + Gerund	recommends starting medication
prescribe	V + NP	prescribes antibiotics
administer	V + NP	administers medication
schedule	V + NP	schedules a visit
diagnose	V + NP	diagnoses pneumonia
refer	V + NP + PP	refers patient to specialist
advise	V + NP + Infinitive	advises patient to rest

This prevents incorrect semantic relationships.

2. Real-time processing

Use a pipeline such as:

Report Input



Streaming Preprocessor



Sentence Detection



Incremental Parser



Medical NER



Semantic Analysis



Clinical Information Extraction



Hospital Database

Techniques

- incremental parsing
- chart parsing
- caching
- parallel processing
- batch processing for historical reports
- streaming for new reports
- GPU acceleration for neural components
- domain-specific dictionaries
- model quantization where appropriate

3. Accuracy improvement

Use multiple validation levels:

Lexical Validation



Syntactic Validation



Agreement Validation



Semantic Validation



Medical Entity Validation



Clinical Output Validation

Recommended evaluation metrics:

Metric	Purpose
Precision	Correct extracted information
Recall	Coverage of important information
F1-score	Overall extraction quality
Parsing accuracy	Correct syntactic structures
Entity F1	Medical entity extraction
Relation F1	Doctor–action–patient relationships
Latency	Real-time performance
Throughput	Reports processed per second

Example target metrics

For a hospital deployment, reasonable engineering targets could be:

Metric	Target
Medical entity F1	≥ 0.90
Relation extraction F1	≥ 0.85
Syntactic parsing accuracy	$\geq 90\%$
Intent/action extraction	$\geq 90\%$
Average processing latency	< 1 second/sentence
System availability	$\geq 99.9\%$

These are **target values for system design**, not guaranteed outcomes.

Final comparison

Feature	CFG	PCFG	Feature Structures	Earley
Basic syntax	✓✓✓	✓✓✓	✓✓✓	✓✓✓
Ambiguity resolution	✗	✓✓✓	✓✓	✓✓✓
Probability	✗	✓✓✓	✗	Can integrate
Agreement	✗	Limited	✓✓✓	✓
Partial input	✗	Limited	✓	✓✓✓
Left recursion	Problematic in naive top-down use	Depends on parser	Depends on parser	✓
Long sentences	Limited	Better	Better	✓✓✓
Real-time suitability	Low	Medium/High	High	High
Medical/financial domain	Limited alone	Good	Excellent	Excellent

Conclusion

CFG is useful as the basic syntactic foundation, but it is insufficient by itself for modern conversational and industry NLP systems. PCFG improves ambiguity resolution through probabilities, feature structures enforce grammatical agreement and additional constraints, while Earley parsing efficiently maintains multiple possible parses and handles incomplete or recursive structures. Combining these techniques with semantic analysis, subcategorization frames, medical/domain ontologies, and efficient chart parsing produces a more accurate, scalable, and real-time system suitable for banking, voice-assistant, and healthcare applications.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Professionalism	Sub. Total	Total %
1								
2								
3								

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____